

Atharva Naveen-Privion

The application includes-

Project One Page Description

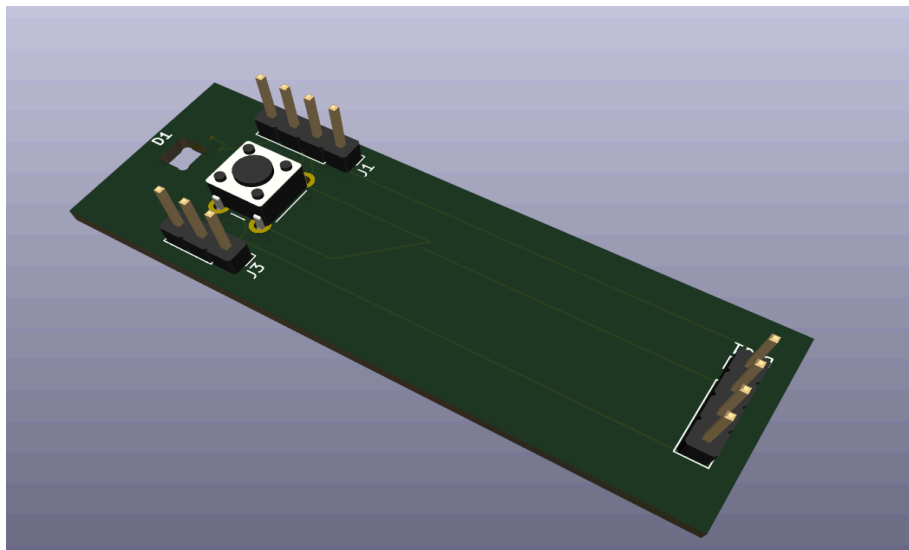
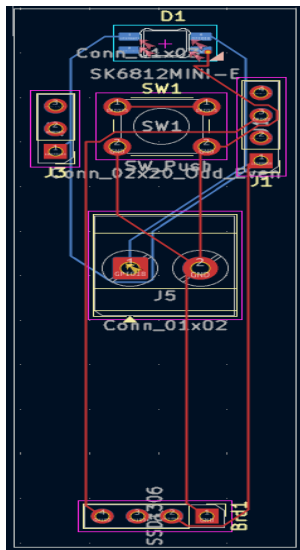
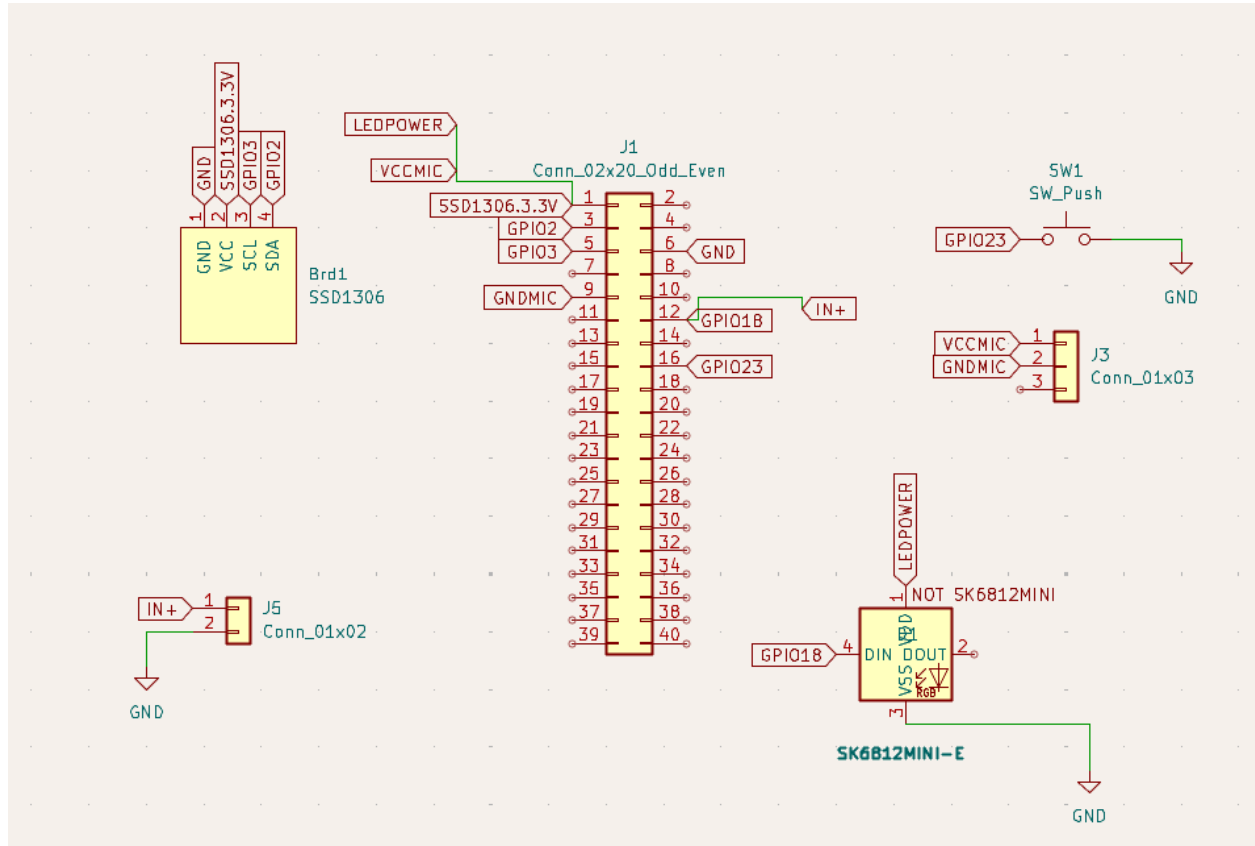
CAD Designs

Model Software Code (HTML)

Project One Page Description-Privion

Privion is a privacy tool built around the idea that people should have a real, physical way to protect themselves from surveillance. Instead of relying on an app or a software setting buried in a menu, this is an actual device you can put on a desk or carry around that actively works to keep your conversations and surroundings private. The device creates active noise cancellation by essentially listening to whatever sound is happening around it and then producing the exact opposite of that sound at the same time. It starts with the microphone, which is constantly picking up audio from the surrounding environment. Once it detects that something is worth canceling, which is determined by the sensitivity threshold you set, it pulls that audio signal in and analyzes the waveform. A waveform is basically just the shape of a sound wave, the peaks and valleys that represent how the air is moving when something makes noise. From there the system calculates what is called an anti-phase signal. Sound travels in waves, and if you take that same wave and flip it upside down so that every peak becomes a valley and every valley becomes a peak, you get its opposite. When the original wave and the flipped wave meet in the air at the same time, they cancel each other out. The pressure changes balance to zero and the sound effectively disappears. The device then plays that flipped signal out through its speaker in real time. The timing has to be close enough that the two signals overlap in the space around the device, which is why the cancellation gain control matters. That setting lets you adjust how strong the output signal is so it actually matches the volume of what it is trying to cancel rather than being too weak or too overpowering. The spectrogram on the interface shows you the audio it is picking up so you can see when it is actively working, and the system keeps doing this continuously as long as it is running. The hardware is built on a custom PCB designed around a Raspberry Pi, and it includes a small OLED display, an RGB LED indicator, a push button, and a microphone. The board itself is compact enough to be practical in everyday spaces like classrooms or offices, which was kind of the whole point of making it a physical device in the first place. The audio protection side of it works through active noise cancellation. The microphone picks up sound in the environment, the system processes it and generates an inverted version of that signal, and then plays it back through a speaker to cancel out the original audio. The idea is that any nearby recording device would have a much harder time picking up a clean signal. You can tune how sensitive it is and how strong the cancellation output is through the onboard controls, so it is not a one size fits all setup. There is also a camera component that uses OpenCV to scan the area for unfamiliar faces or objects that could be recording devices. It gives each situation a threat score and highlights anything suspicious on screen so you know what it is reacting to. One thing that was important to the design is that all of the data stays on the device. No biometric information gets sent anywhere or saved after you are done using it, which keeps the tool in line with the privacy values it was built around.

CAD Designs



Sample Code for an online HTML Version

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title> - Basic Edition</title>
<style>
body { font-family: sans-serif; padding: 20px; line-height: 1.5; }
canvas { background: #000; display: block; margin: 10px 0; width: 100%; height: 200px; }
.controls { margin: 20px 0; border: 1px solid #ccc; padding: 15px; }
.metrics { display: flex; gap: 20px; font-weight: bold; margin-bottom: 10px; }
#logBox { height: 100px; overflow-y: auto; background: #eee; padding: 10px; font-family: monospace; font-size: 12px; }
.active-text { color: red; }
</style>
</head>
<body>

<h1></h1>

<div class="metrics">
  <div>Status: <span id="lblStatus">IDLE</span></div>
  <div>Activity: <span id="lblActivity">SILENCE</span></div>
  <div>Room Noise: <span id="mRMS">-80 dB</span></div>
</div>

<button id="engageBtn" onclick="toggleSystem()" style="padding: 10px 20px; font-size: 16px;">ARM SYSTEM</button>

<canvas id="vizCanvas"></canvas>

<div class="controls">
  <div>
    <label>Trigger Sensitivity (Threshold): </label>
    <input type="range" id="thresholdSlider" min="-80" max="-10" value="-45" oninput="updateThreshold(this)">
    <span id="thresholdVal">-45 dB</span>
  </div>
  <br>
  <div>
    <label>Cancellation Gain: </label>
    <input type="range" id="gainSlider" min="0" max="2" step="0.01" value="1.0" oninput="updateGain(this)">
    <span id="gainVal">1.00x</span>
  </div>
</div>

<div class="metrics">
  <div>Trigger: <span id="mState">STANDBY</span></div>
  <div>Output: <span id="mOut">0.00x</span></div>
</div>

<div id="logBox">System ready...</div>

<script>
let audioCtx, micStream, micSource, analyser, gainNode, isArmed = false, isTriggered = false;
let threshold = -45;
let userGain = 1.0;

function log(msg) {
  const box = document.getElementById('logBox');
  box.innerHTML = `[${new Date().toLocaleTimeString()}] ${msg}<br>` + box.innerHTML;
}

async function toggleSystem() {
  if (!isArmed) {
    try {
      micStream = await navigator.mediaDevices.getUserMedia({ audio: { echoCancellation: false, noiseSuppression: false } });
      audioCtx = new (window.AudioContext || window.webkitAudioContext)();
      micSource = audioCtx.createMediaStreamSource(micStream);
      analyser = audioCtx.createAnalyser();
      analyser.fftSize = 1024;

      gainNode = audioCtx.createGain();
      gainNode.gain.value = 0;

      micSource.connect(analyser);
      micSource.connect(gainNode);
      gainNode.connect(audioCtx.destination);

      isArmed = true;
```

```

    document.getElementById('engageBtn').textContent = 'SYSTEM ACTIVE (CLICK TO RESET)';
    document.getElementById('lblStatus').textContent = 'MONITORING';
    log("System Armed.");
    monitor();
    draw();
  } catch(e) { alert("Mic access required."); }
} else {
  location.reload();
}
}

```

```

function updateThreshold(el) {
  threshold = parseInt(el.value);
  document.getElementById('thresholdVal').textContent = threshold + " dB";
}

```

```

function updateGain(el) {
  userGain = parseFloat(el.value);
  document.getElementById('gainVal').textContent = userGain.toFixed(2) + "x";
}

```

```

function monitor() {
  if (!isArmed) return;
  requestAnimationFrame(monitor);

```

```

  const data = new Float32Array(analyser.fftSize);
  analyser.getFloatTimeDomainData(data);

```

```

  let sum = 0;
  for (let i = 0; i < data.length; i++) sum += data[i] * data[i];
  const rms = Math.sqrt(sum / data.length);
  const db = rms > 0 ? 20 * Math.log10(rms) : -80;

```

```

  document.getElementById('mRMS').textContent = `${db.toFixed(1)} dB`;

```

```

  if (db > threshold) {
    if (!isTriggered) {
      isTriggered = true;
      gainNode.gain.setTargetAtTime(-userGain, audioCtx.currentTime, 0.01);
      updateUI(true);
      log("Triggered: Countermeasure active.");
    }
  } else {
    if (isTriggered) {
      isTriggered = false;
      gainNode.gain.setTargetAtTime(0, audioCtx.currentTime, 0.1);
      updateUI(false);
      log("Standby: Silence detected.");
    }
  }
}

```

```

function updateUI(active) {
  const lbl = document.getElementById('lblActivity');
  const state = document.getElementById('mState');
  const out = document.getElementById('mOut');

```

```

  if (active) {
    lbl.textContent = "AUDIO DETECTED";
    lbl.className = "active-text";
    state.textContent = "TRANSMITTING";
    out.textContent = userGain.toFixed(2) + "x";
  } else {
    lbl.textContent = "SILENCE";
    lbl.className = "";
    state.textContent = "STANDBY";
    out.textContent = "0.00x";
  }
}

```

```

function draw() {
  if (!isArmed) return;
  requestAnimationFrame(draw);
  const canvas = document.getElementById('vizCanvas');
  const ctx = canvas.getContext('2d');
  const W = canvas.width = canvas.offsetWidth;
  const H = canvas.height = canvas.offsetHeight;

```

```

  const buffer = new Float32Array(analyser.frequencyBinCount);
  analyser.getFloatTimeDomainData(buffer);

```

```

  ctx.clearRect(0, 0, W, H);
  ctx.beginPath();
  ctx.strokeStyle = isTriggered ? '#ff0000' : '#00ff00';
  ctx.lineWidth = 1;

```

```

  for (let i = 0; i < buffer.length; i++) {
    const x = (i / buffer.length) * W;
    const y = (0.5 + buffer[i] * 0.5) * H;

```

```
i === 0 ? ctx.moveTo(x, y) : ctx.lineTo(x, y);
}
ctx.stroke();

// Threshold Line (Simple representation)
const threshPos = H - ((threshold + 80) / 70) * H;
ctx.strokeStyle = 'yellow';
ctx.beginPath(); ctx.moveTo(0, threshPos); ctx.lineTo(W, threshPos); ctx.stroke();
}
</script>
</body>
</html>
```